Is this coin fair?

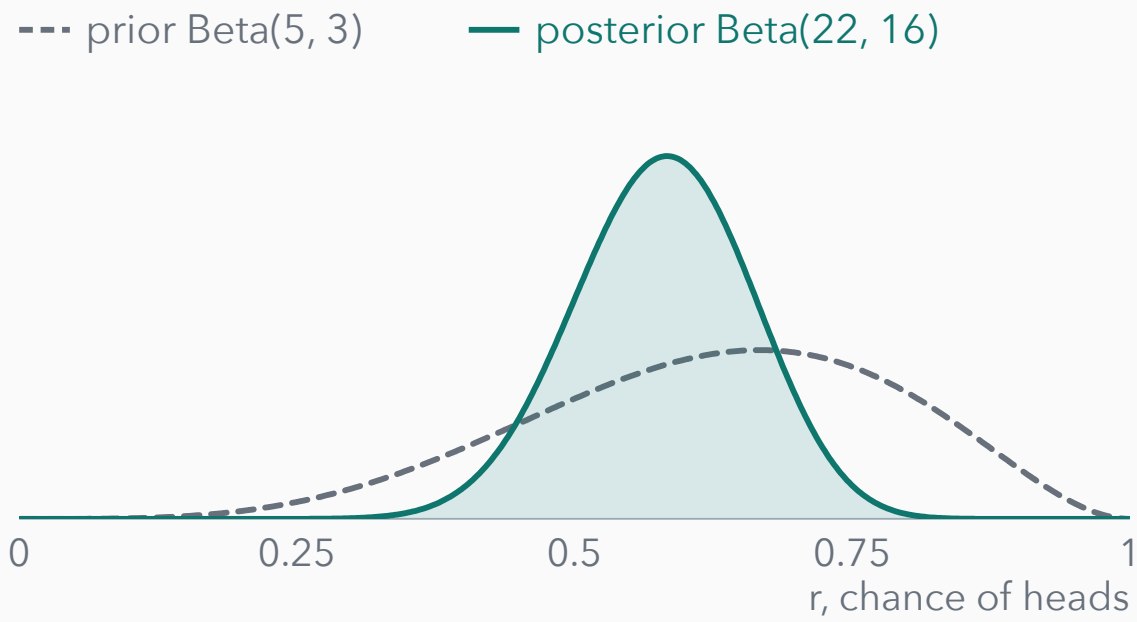
H H T H T H T H H H T T H T H

H T H T T H H H T T H T H T H

An honest answer is a distribution over r, the coin's chance of heads: where it probably is, and how sure we can be.

> "Probably fair, but I can't be sure" is a distribution over r. Bayes' rule gives it.

With a Beta prior, Bayes' rule has a formula



$$p(r \mid \mathcal{D}) = \frac{p(\mathcal{D} \mid r)\, p(r)}{p(\mathcal{D})}$$

$$\mathrm{Beta}(5, 3) \xrightarrow{17\ \mathrm{H},\ 13\ \mathrm{T}} \mathrm{Beta}(22, 16)$$

$$p(\mathcal{D}) = \frac{B(22, 16)}{B(5, 3)}$$

$$\log p(\mathcal{D}) = -21.40$$

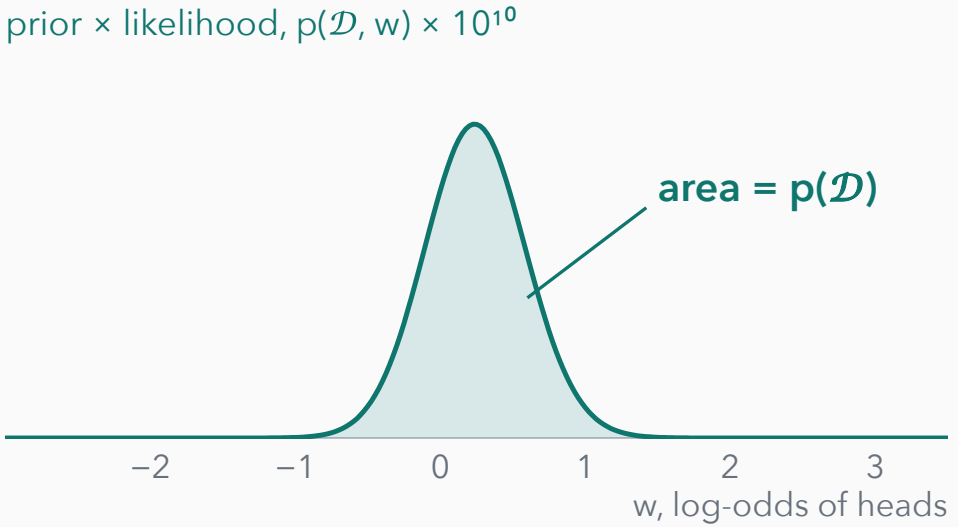
> Even the normaliser p($\mathcal{D}$), the evidence, has a formula. That is luck: this prior and this likelihood are conjugate.

Change the prior, and the formula disappears

$$w \sim \mathcal{N}(0, 1), \qquad r = \mathrm{sigmoid}(w)$$

$$p(\mathcal{D}) = \int \mathcal{N}(w; 0, 1)\, r(w)^{17} \big(1 - r(w)\big)^{13}\, dw$$

On a 200,001-point grid: area $\approx 4.1 \times 10^{-10}$, so $\log p(\mathcal{D}) \approx -21.62$.



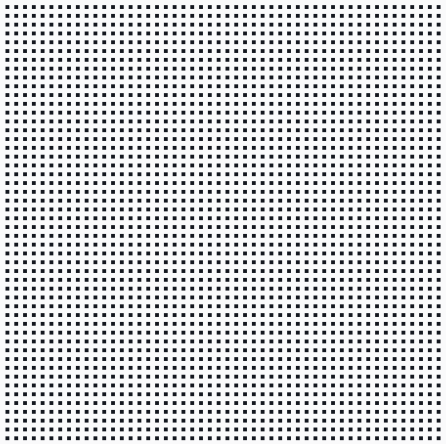
> The evidence is this shaded area, and no formula gives it. With one unknown, a grid of points still can.

A grid works for one unknown, not for a million

1 unknown

100 points

2 unknowns



100 × 100 = 10,000 points

A small network: 1,000,000 weights.

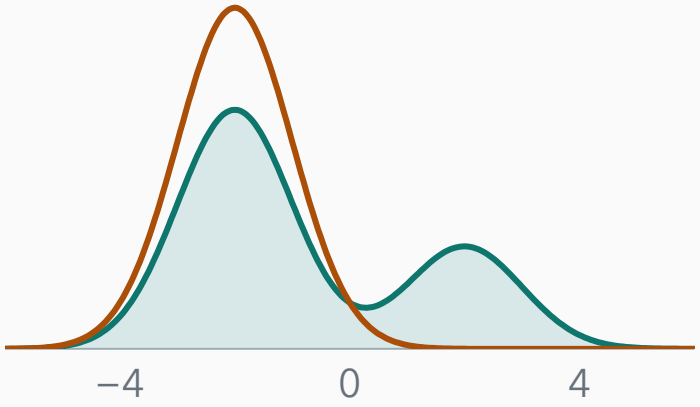
$10^{2{,}000{,}000}$

grid points, at 100 per weight. The observable universe has about 10^{80} atoms.

A small neural network has a million unknowns: 10^2,000,000 grid points. We need a different idea.

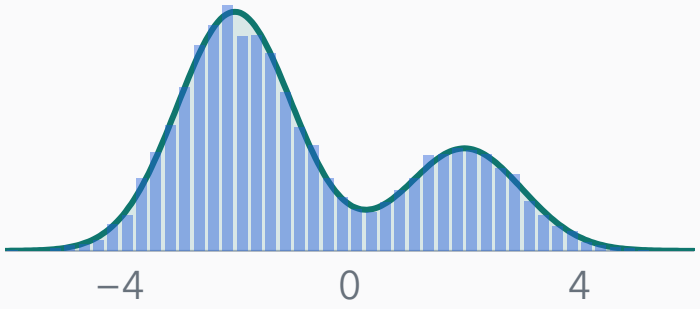
Three ways around an integral nobody can do

Laplace



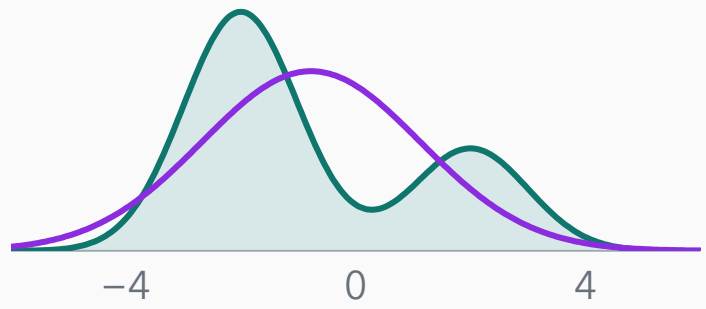
A Gaussian at the highest peak: N(−2.0, 1.0²). It never sees the second bump.

MCMC



20,000 random-walk steps. Their mean, −0.80, matches the true −0.80.

Variational inference



The closest Gaussian, found by optimisation: N(−0.78, 1.90²).

> Variational inference: choose the closest member of a simple family, by optimisation. That is today.

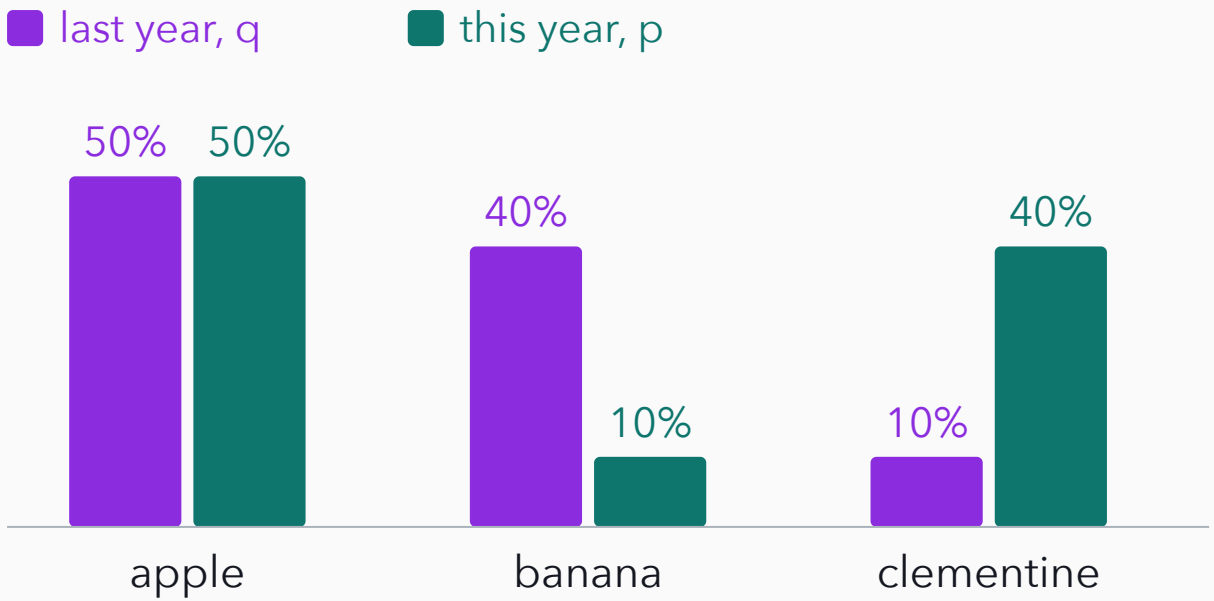
The plan: push a simple q towards the posterior

$$q^* = \arg\min_{q \in \mathcal{Q}} \ \text{distance}\big(q,\ p(u \mid \mathcal{D})\big)$$



> Five obstacles stand in the way. Each act removes one; then we return to this coin.

Two surveys: how different are they?



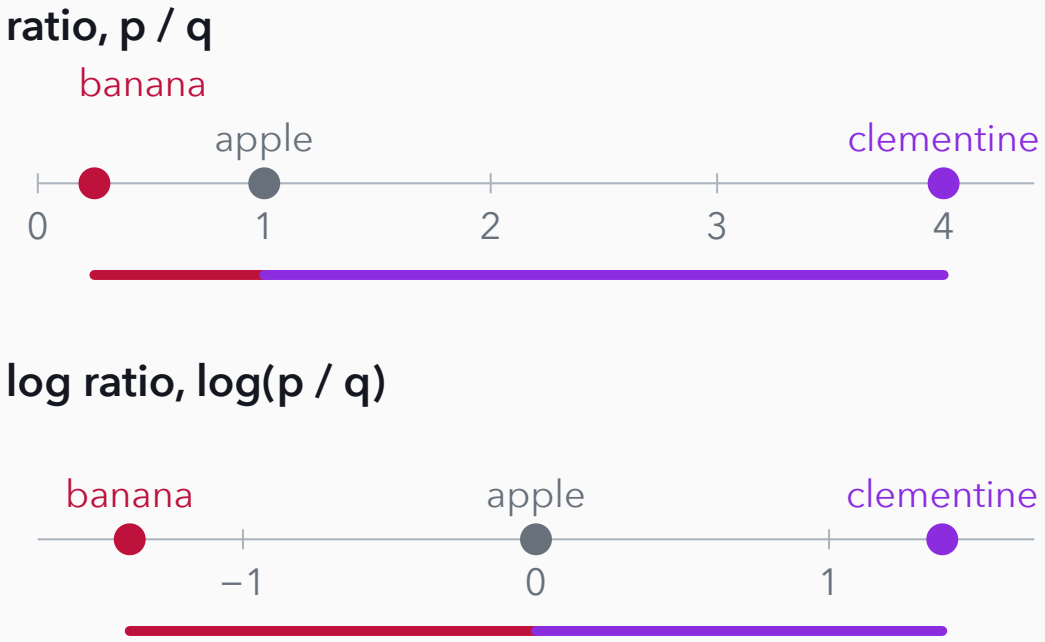
Favourite fruit in a class, last year (q) and this year (p).

Bananas fell from 40% to 10%.
Clementines rose from 10% to 40%.

> A fourfold fall and a fourfold rise. Should they count the same?

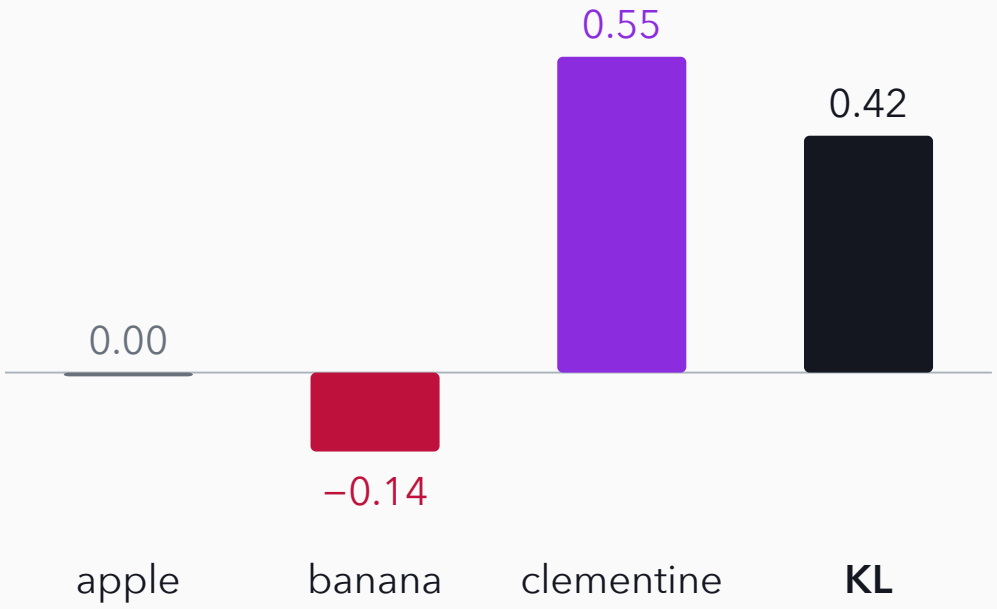
Ratios say “how many times”; logs make up and down cancel

fruit	p	q	p/q	$\log(p/q)$
apple	0.5	0.5	1	0.00
banana	0.1	0.4	¼	−1.39
clementine	0.4	0.1	4	1.39



Take logs: log 4 = +1.39 and log ¼ = −1.39. Equal and opposite.

Weight by how common each fruit is now: that is KL



$$\mathrm{KL}(p \,\|\, q) = \sum_x p(x) \, \log \frac{p(x)}{q(x)}$$

$$= 0.5(0) + 0.1(-1.39) + 0.4(1.39)$$

$$\approx 0.42 \text{ nats}$$

> The rare banana barely counts; the common clementine dominates. Total 0.42 nats: the Kullback–Leibler divergence.

KL is the extra message length of using the wrong code

weather	p	q	code built for p	code built for q
sunny	0.50	0.25	`0`	`10`
cloudy	0.25	0.50	`10`	`0`
rainy	0.25	0.25	`11`	`11`

$$\text{average length under } p: \quad \text{code for } p = 1.50 \text{ bits}, \qquad \text{code for } q = 1.75 \text{ bits}$$

$$1.75 - 1.50 = 0.25 \text{ bits} = \mathrm{KL}(p \| q) \text{ with } \log_2$$

> A code built for the wrong frequencies q needs 1.75. The extra 0.25 bits a day are exactly KL(p‖q), in base 2.

You have minimised KL before: in every cross-entropy loss

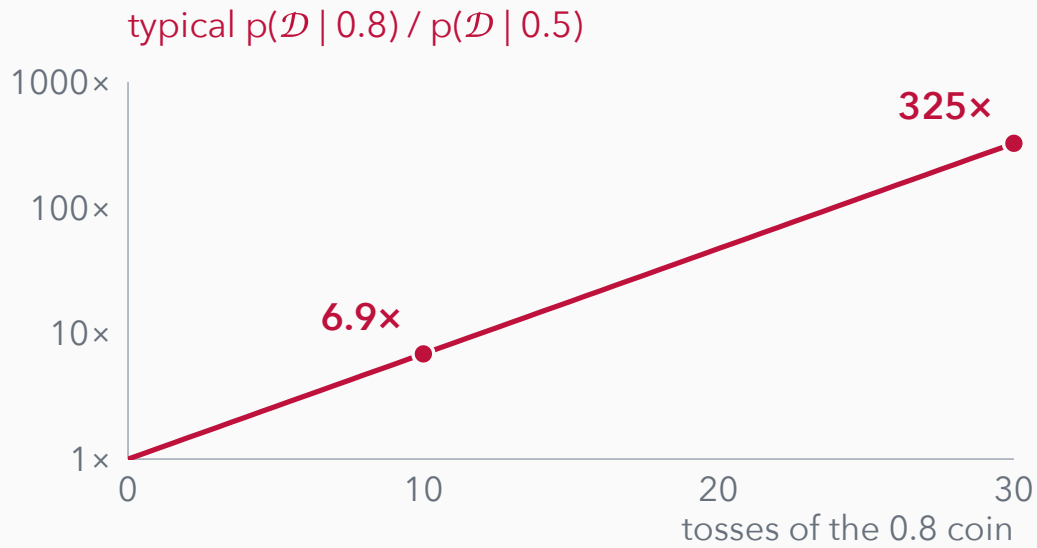
$$\underbrace{-\sum_x p(x)\log q(x)}_{\text{cross-entropy } H(p,q)} = \underbrace{-\sum_x p(x)\log p(x)}_{\text{entropy } H(p)} + \mathrm{KL}(p\|q)$$

$$\text{classifier:} \quad \min_\theta H(y,\, q_\theta) \iff \min_\theta \mathrm{KL}(y \,\|\, q_\theta)$$

$$\text{maximum likelihood:} \quad -\tfrac{1}{N}\textstyle\sum_i \log p_\theta(x_i) \;\approx\; H(p_{\text{data}}) + \mathrm{KL}(p_{\text{data}} \,\|\, p_\theta)$$

> Maximum likelihood minimises KL(data ‖ model). Variational inference is the third use.

A KL value is how fast evidence piles up



$$\mathrm{KL}\big(\mathrm{Bern}(0.8) \,\|\, \mathrm{Bern}(0.5)\big) \approx 0.19 \text{ nats}$$

$$\frac{p(\mathcal{D} \mid 0.8)}{p(\mathcal{D} \mid 0.5)} \approx e^{0.19\,n}$$

The reverse direction, KL(0.5‖0.8) = 0.22, is a different number.

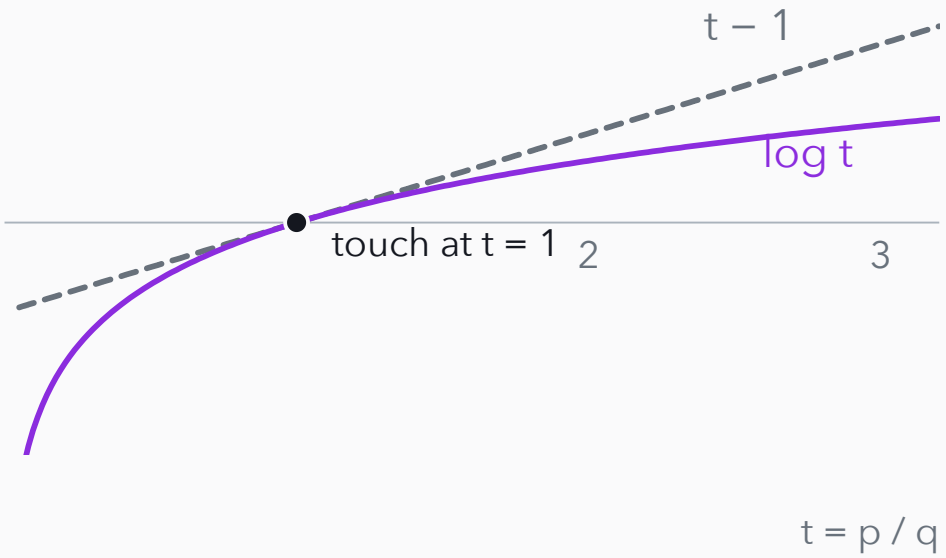
> About e^(30 × 0.19) ≈ 325. KL is the rate at which evidence piles up: 6.9× after 10 tosses, 325× after 30.

For densities, sums become integrals, and KL ≥ 0

$$\mathrm{KL}(q \,\|\, p) = \int q(u)\, \log \frac{q(u)}{p(u)}\, du$$

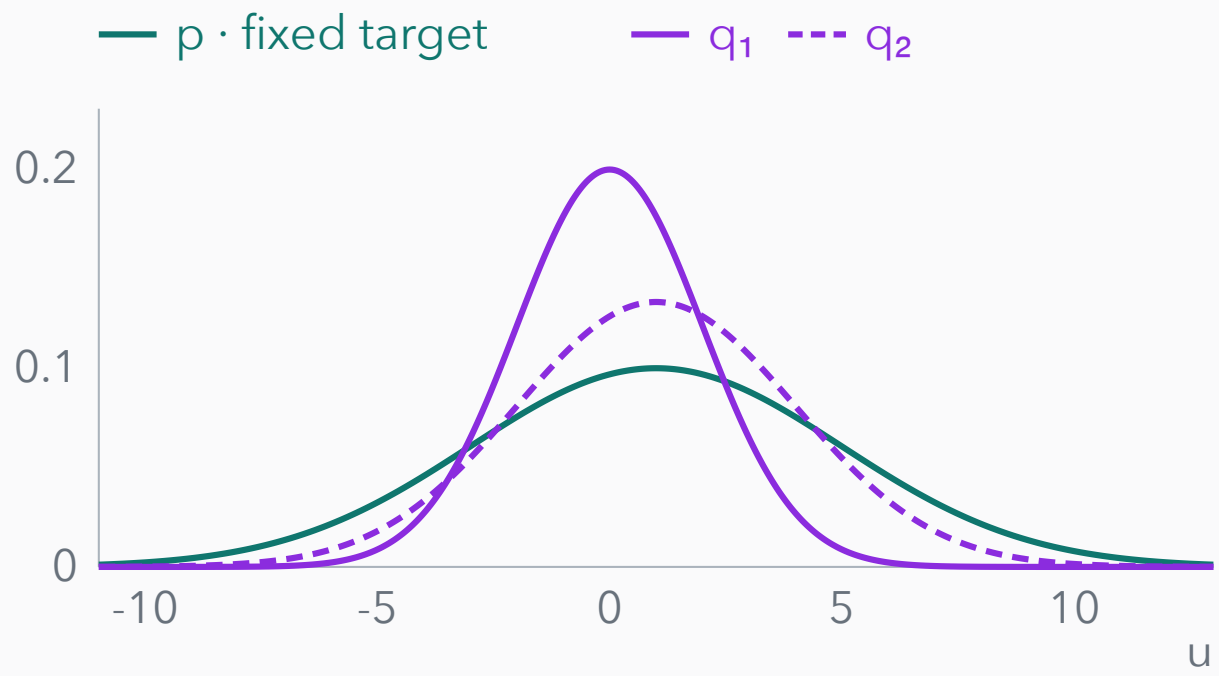
$$-\mathrm{KL}(q\|p) = \mathbb{E}_q\left[\log \frac{p}{q}\right] \le \mathbb{E}_q\left[\frac{p}{q} - 1\right]$$

$$= \int p\, du - 1 = 0$$



log t never rises above its tangent t − 1. Inside the average that gives KL ≥ 0, with 0 only when q = p.

Which Gaussian is closer to p?



$$p = \mathcal{N}(1, 4^2)$$

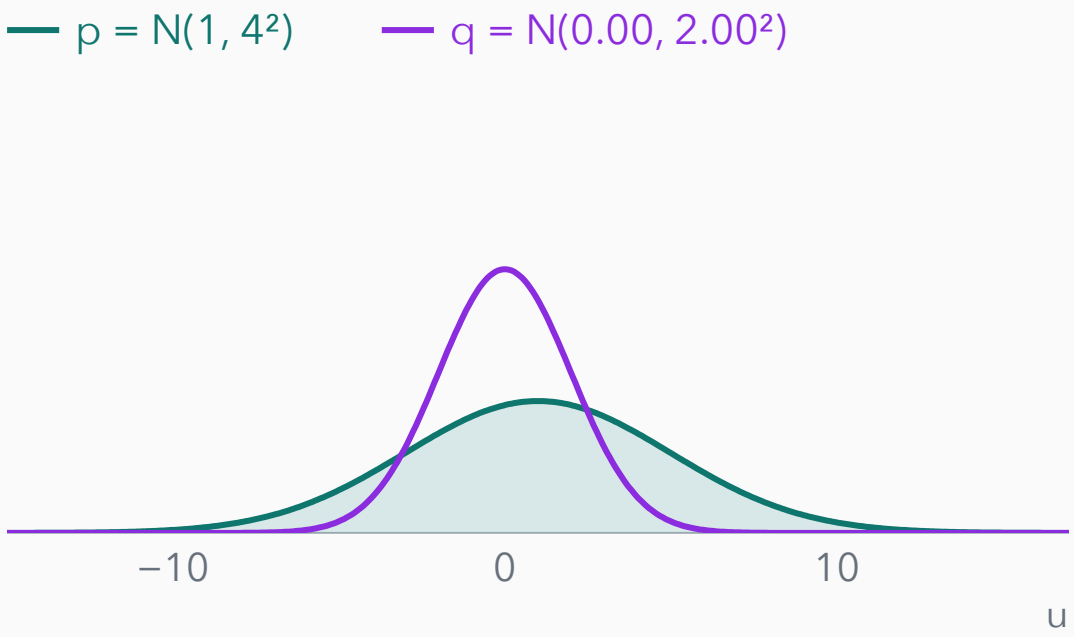
$$q_1 = \mathcal{N}(0, 2^2)$$

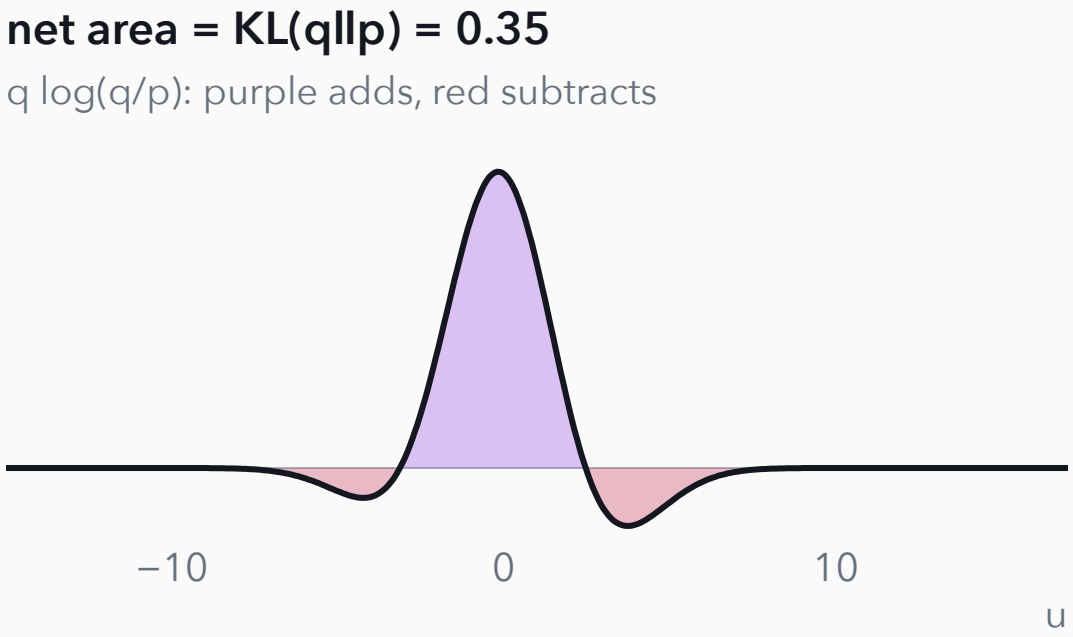
$$q_2 = \mathcal{N}(1, 3^2)$$

	$\mathrm{KL}(q\|\|p)$
q_1	0.35
q_2	0.07

KL(q_1‖p) = 0.35 nats and KL(q_2‖p) = 0.07. The vote now has a number.

KL is an area you can watch





Purple area adds and red area subtracts; the net area is the KL. Drag q, or flip the direction.

VI uses KL(q‖p) because it needs only what we have

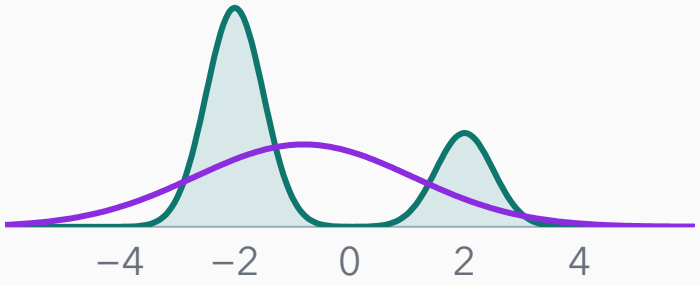
divergence	needs	for a posterior p(u | $\mathcal{D}$)
$\mathrm{KL}(p \,\|\, q)$	samples from p	impossible: that is the problem
$\mathrm{KL}(q \,\|\, p)$	samples from q; log p up to a constant	possible: we choose q
Wasserstein, TV	samples from p, or p normalised	impossible

The price: $q(u) \log \frac{q(u)}{p(u)}$ blows up wherever q puts mass and p is near zero, so q avoids those places.

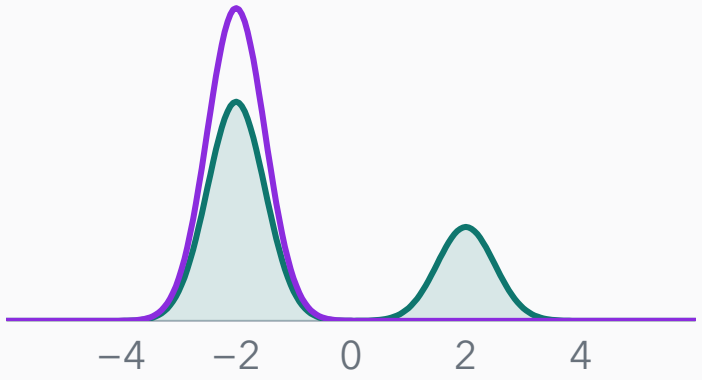
> KL(q‖p) needs only what we can get. The price: q avoids any region where p is near zero.

The price: reverse KL picks one bump

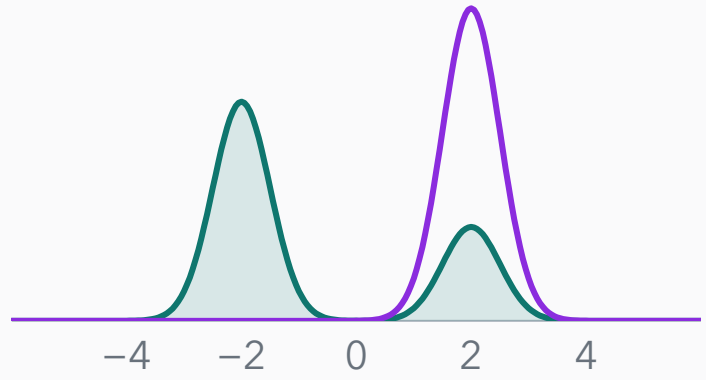
KL(p||q) 0.72



KL(q||p), start left 0.36



KL(q||p), start right 1.20



KL(p||q) spreads over both bumps. KL(q||p) locks onto whichever bump is nearer its start, and reports it confidently.

A score picks a winner. Can it improve a loser?

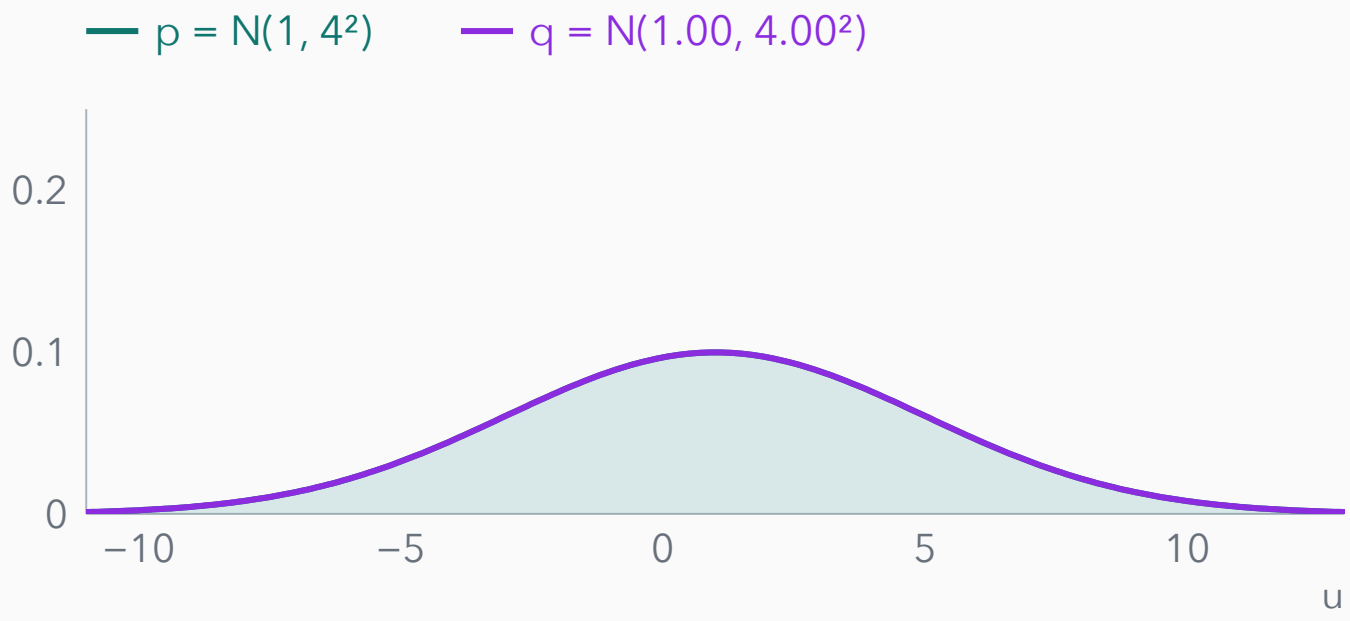

```
mu  = Parameter(tensor(4.0))
rho = Parameter(tensor(log(0.5)))
for step in range(200):
    q = Normal(mu, rho.exp())
    loss = kl_divergence(q, p)
    opt.zero_grad()
    loss.backward()
    opt.step()
```

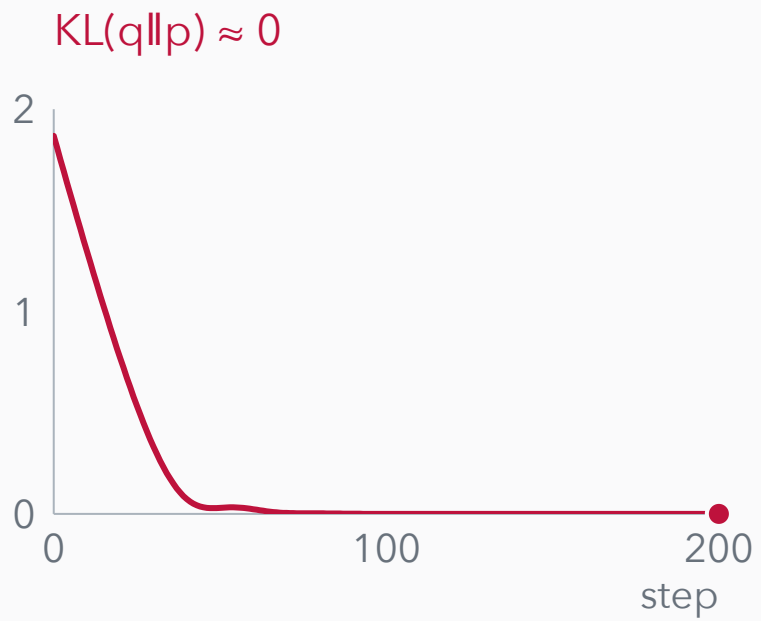
$$\psi^* = \arg\min_{\psi} \mathrm{KL}(q_\psi \,\|\, p)$$

$$\sigma = \exp(\rho) > 0 \ \text{ for every } \rho$$

Store ρ = log σ: whatever an update does to ρ, σ = exp(ρ) stays positive.

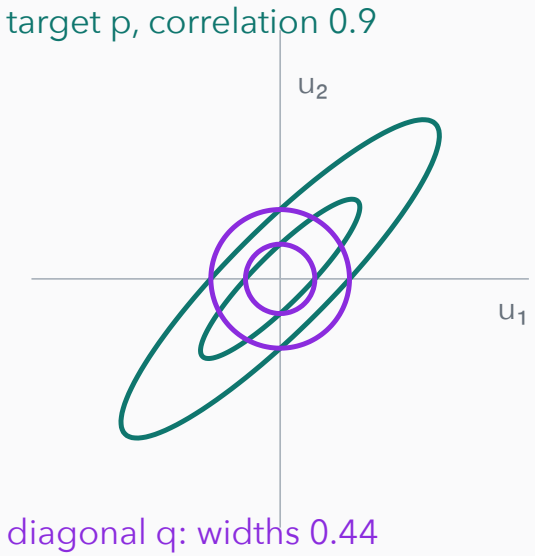
Press play: q slides onto p

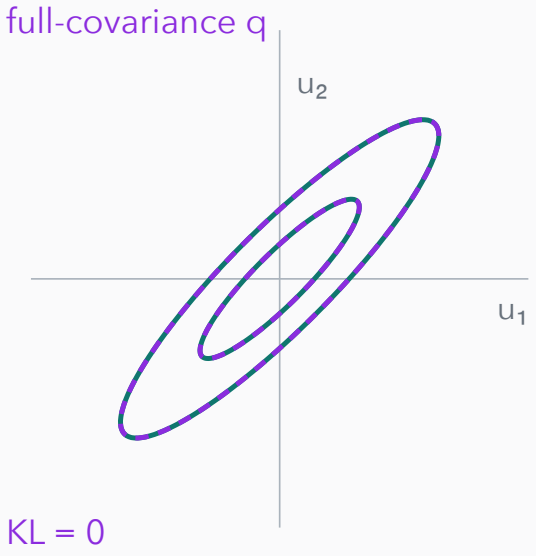




> The width gets there first and overshoots a little; the centre slides in by step 100. Then KL ≈ 0.

In 2-D, a q that cannot tilt shrinks instead





$$p = \mathcal{N}\left(0, \begin{bmatrix} 1 & 0.9 \\ 0.9 & 1 \end{bmatrix}\right)$$

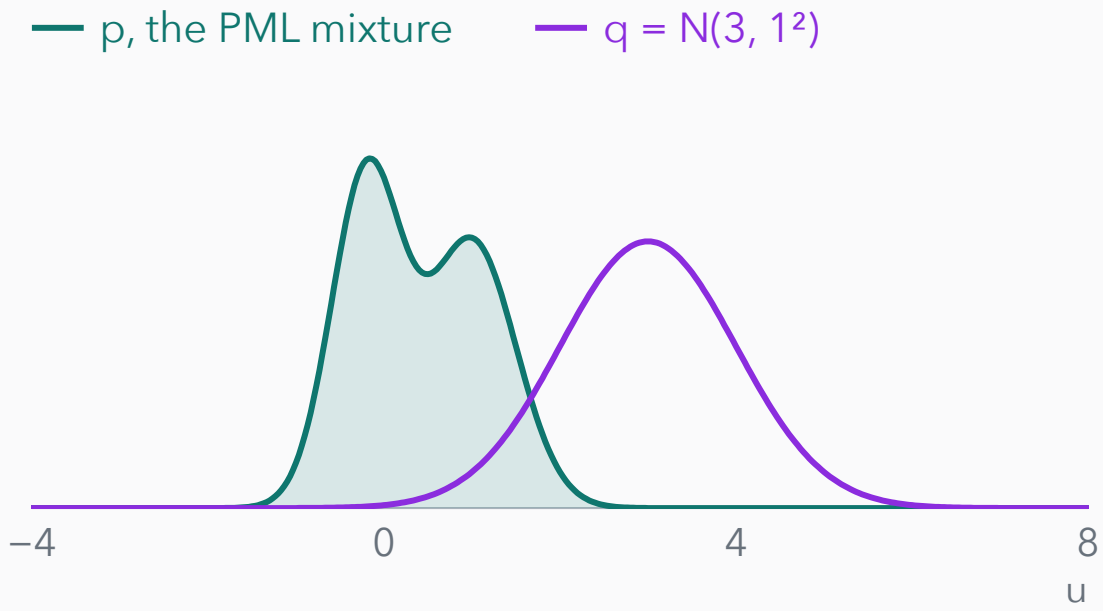
$$q = \mathcal{N}\left(0, \mathrm{diag}(s^2, s^2)\right)$$

$$s = 0.44, \quad \mathrm{KL} = 0.83$$

$$q = \mathcal{N}(0, LL^\top) : \ \mathrm{KL} = 0$$

> Let q tilt with a full covariance, Σ = LLᵀ, and the KL drops to 0.

PyTorch has no formula for this KL. The definition still works.




```
kl_divergence(q, mixture)
```

```
NotImplementedError:
No KL(p || q) is implemented
for p type Normal and
q type MixtureSameFamily
```

No formula. But KL is an average under q, and averages can be estimated by sampling.

Draw from q, average the log ratio

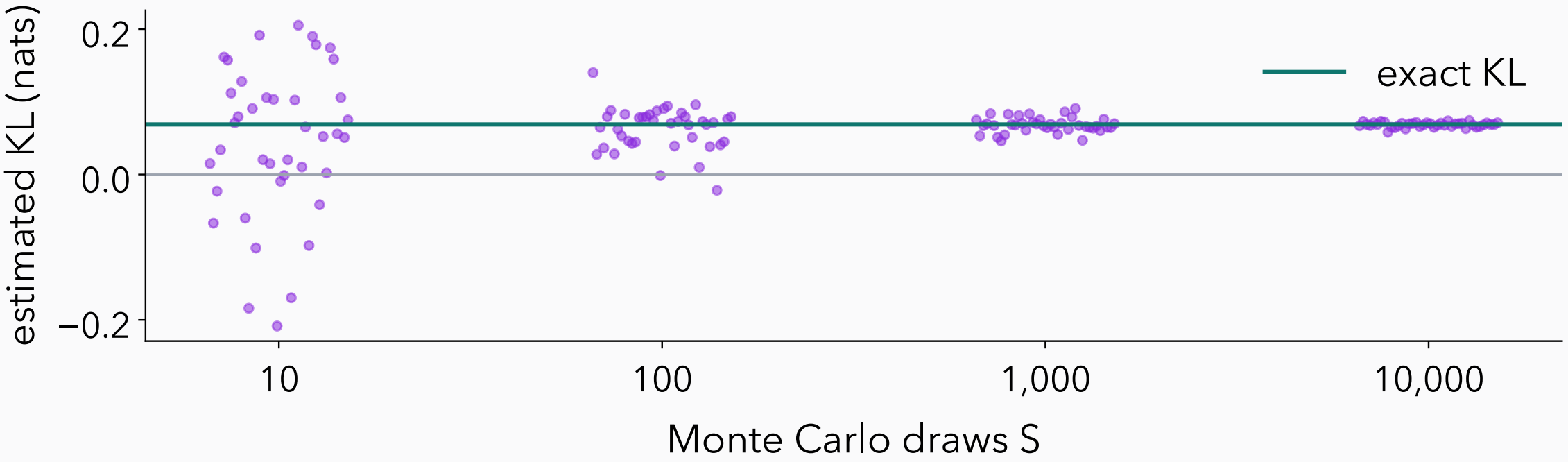
$$\mathrm{KL}(q\|p) = \mathbb{E}_{u\sim q}\big[\log q(u) - \log p(u)\big] \approx \frac{1}{S}\sum_{i=1}^{S}\big[\log q(u_i) - \log p(u_i)\big]$$

```
u = q.sample((S,))
log_q = q.log_prob(u)
log_p = p.log_prob(u)
est = (log_q - log_p).mean()
```

	KL(q_2‖p)
exact formula	0.069
estimate, S = 10,000	0.067

Check it on a pair we can solve: with 10,000 draws, 0.067 against the exact 0.069.

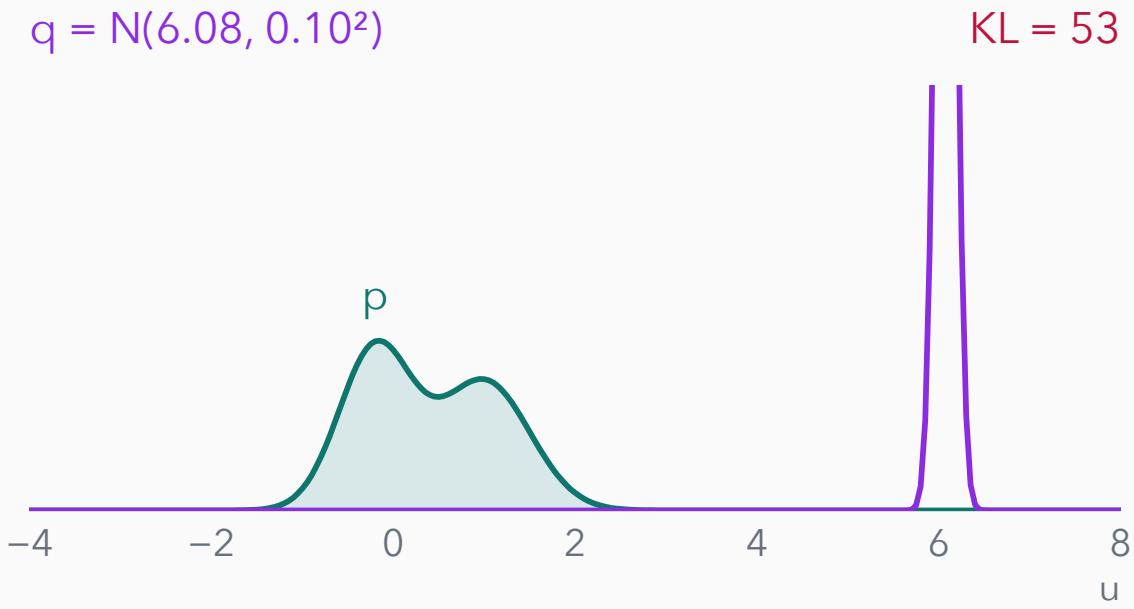
More samples, tighter estimates



Forty repeats at each sample size. The spread shrinks like $1/\sqrt{S}$; the exact value never moves.

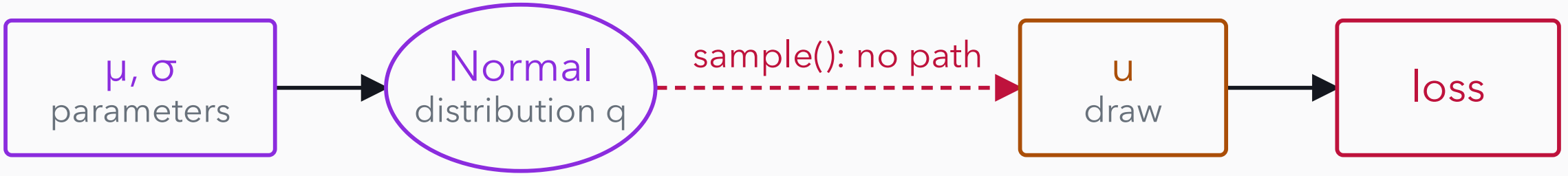
Now optimise it: the code runs, and q does not move


```
u = q.sample((5000,))
log_q = q.log_prob(u)
log_p = p.log_prob(u)
loss = (log_q - log_p).mean()
opt.zero_grad()
loss.backward(); opt.step()
```



No error and no warning, and q stays at N(6, 0.1²). After 400 steps the KL is still about 53.

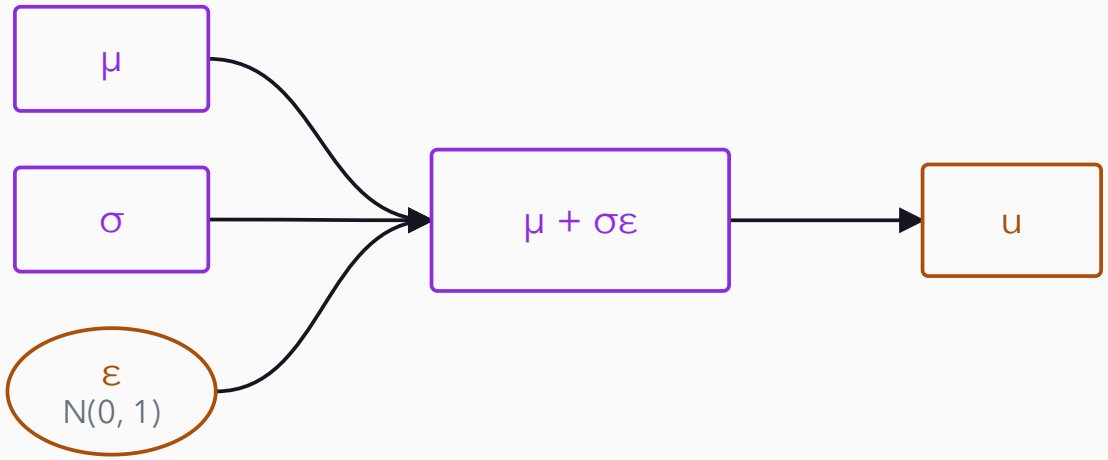
The gradient that would move q never reaches it



	$\partial \mathrm{KL}/\partial \mu$	$\partial \mathrm{KL}/\partial \sigma$
through sample()	0.0013	−0.0003
true gradient	−0.063	−0.375

> What survives averages to zero: 0.0013 and −0.0003, where the true gradient is −0.063 and −0.375.

Move the randomness outside: u = μ + σε



$$\epsilon \sim \mathcal{N}(0,1), \qquad u = \mu + \sigma\epsilon$$

$$\epsilon = -0.75, \quad \mu = 0, \quad \sigma = 2$$

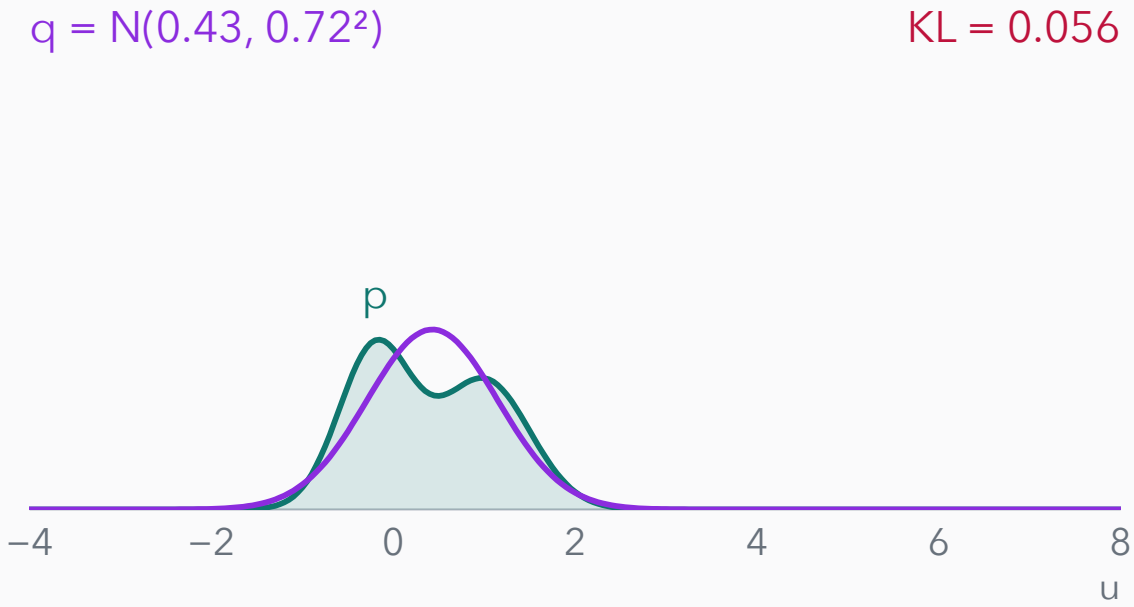
$$u = -1.5$$

$$\frac{\partial u}{\partial \mu} = 1, \qquad \frac{\partial u}{\partial \sigma} = \epsilon = -0.75$$

> u = −1.5, ∂u/∂μ = 1 and ∂u/∂σ = ε = −0.75. The gradient now has a path. In PyTorch this is rsample().

Change one word: rsample() fits the mixture

```
# one word changed
u = q.rsample((5000,))
log_q = q.log_prob(u)
log_p = p.log_prob(u)
loss = (log_q - log_p).mean()
opt.zero_grad()
loss.backward(); opt.step()
```



q settles at N(0.43, 0.72²), KL = 0.056: the best single Gaussian for this mixture. No formula was needed.

The real target is a posterior we cannot evaluate

$$\mathrm{KL}\big(q(u) \,\|\, p(u \mid \mathcal{D})\big) = \mathbb{E}_q\big[\log q(u) - \log p(u \mid \mathcal{D})\big]$$

$$p(u \mid \mathcal{D}) = \frac{p(\mathcal{D}, u)}{p(\mathcal{D})}, \qquad p(\mathcal{D}) = \int p(\mathcal{D}, u)\, du$$

> But p(u | $\mathcal{D}$) = p($\mathcal{D}$, u) / p($\mathcal{D}$), and p($\mathcal{D}$) is the integral from Act 0 that nobody can do.

Substitute Bayes' rule and see what survives

$$\mathrm{KL}\big(q \,\|\, p(u \mid \mathcal{D})\big) = \mathbb{E}_q\big[\log q(u) - \log p(u \mid \mathcal{D})\big]$$

$$= \mathbb{E}_q\big[\log q(u) - \log p(\mathcal{D}, u)\big] + \log p(\mathcal{D})$$

$$\log p(\mathcal{D}) = \underbrace{\mathbb{E}_q\big[\log p(\mathcal{D}, u) - \log q(u)\big]}_{\mathrm{ELBO}(q)} + \mathrm{KL}\big(q \,\|\, p(u \mid \mathcal{D})\big)$$

Rearrange: log p($\mathcal{D}$) = ELBO + KL. The ELBO needs only the joint p($\mathcal{D}$, u) = likelihood × prior.

Maximise the ELBO; the evidence never appears

```
w = q.rsample((256,))
log_q = q.log_prob(w)
elbo = (log_joint(w) - log_q).mean()
opt.zero_grad()
(-elbo).backward(); opt.step()
```

the loop needs	available?
draws from q	yes: we chose q
log q(w)	yes
log p($\mathcal{D}$, w) = log prior + log likelihood	yes

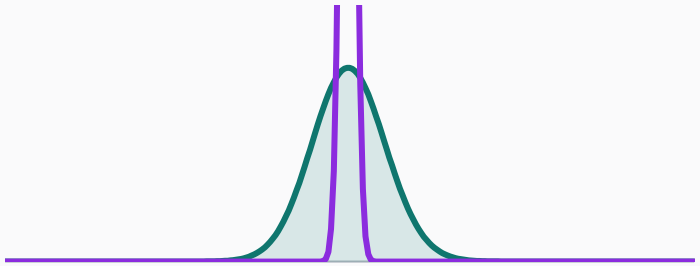
$\log p(\mathcal{D})$ never appears. It does not change with q, so raising the ELBO lowers the KL.

log p($\mathcal{D}$) is constant in q. Maximising the ELBO is minimising the KL to the posterior.

Two forces: sit where the joint is high, stay spread out

$$\mathrm{ELBO}(q) = \underbrace{\mathbb{E}_q\big[\log p(\mathcal{D}, u)\big]}_{\text{sit where the joint is high}} + \underbrace{H(q)}_{\text{stay spread out}}$$

too narrow: σ = 0.06



E_q[log joint] −21.49, H(q) −1.39

ELBO −22.89

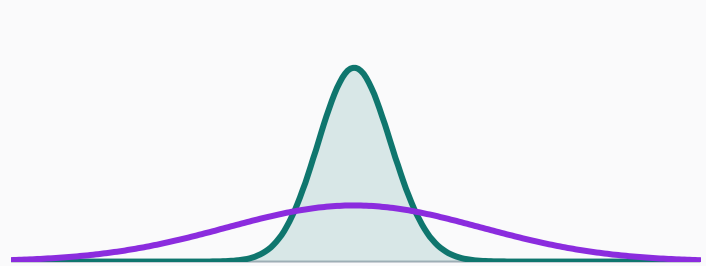
balanced: σ = 0.35



E_q[log joint] −22.00, H(q) 0.38

ELBO −21.62

too wide: σ = 1.20



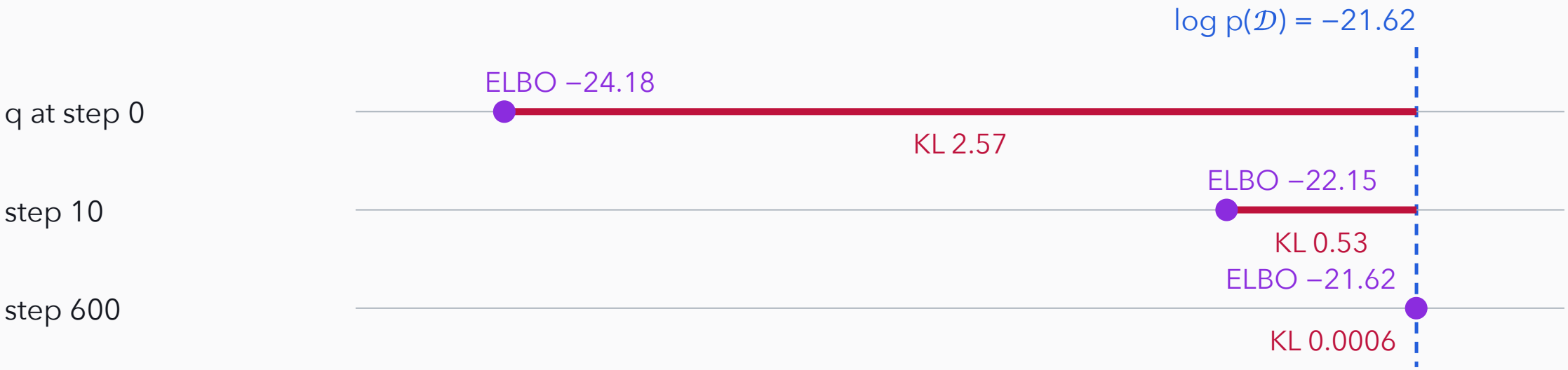
E_q[log joint] −26.86, H(q) 1.60

ELBO −25.26

> Too narrow pays in entropy; too wide wanders into a low joint. The balanced q has the highest ELBO.

One fixed total: raising the ELBO lowers the KL

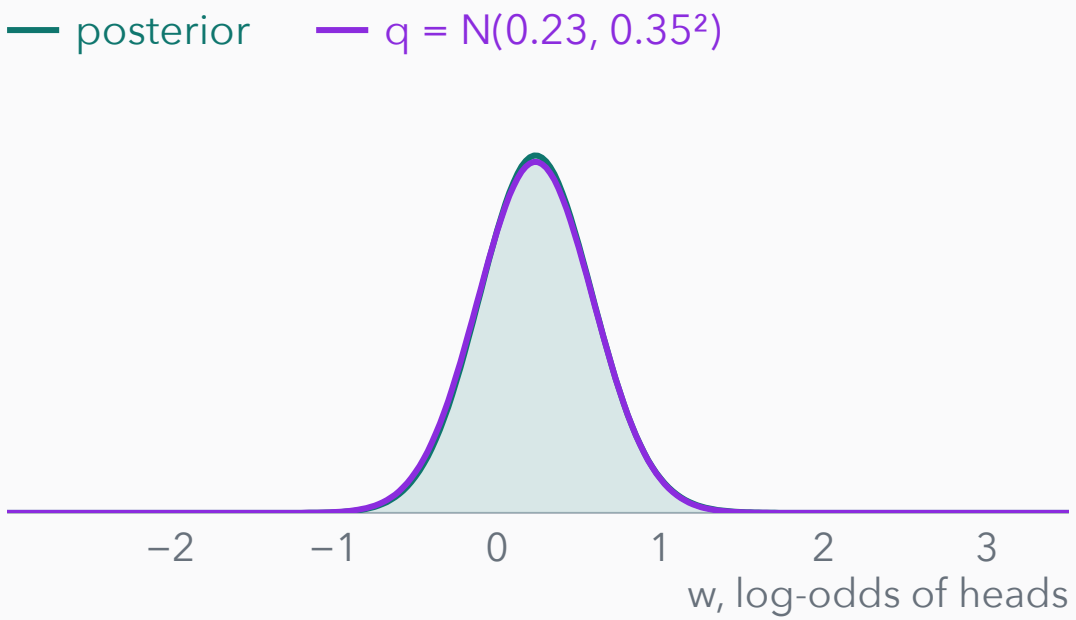
$$\log p(\mathcal{D}) = \mathrm{ELBO}(q) + \mathrm{KL}\big(q \,\|\, p(u \mid \mathcal{D})\big)$$

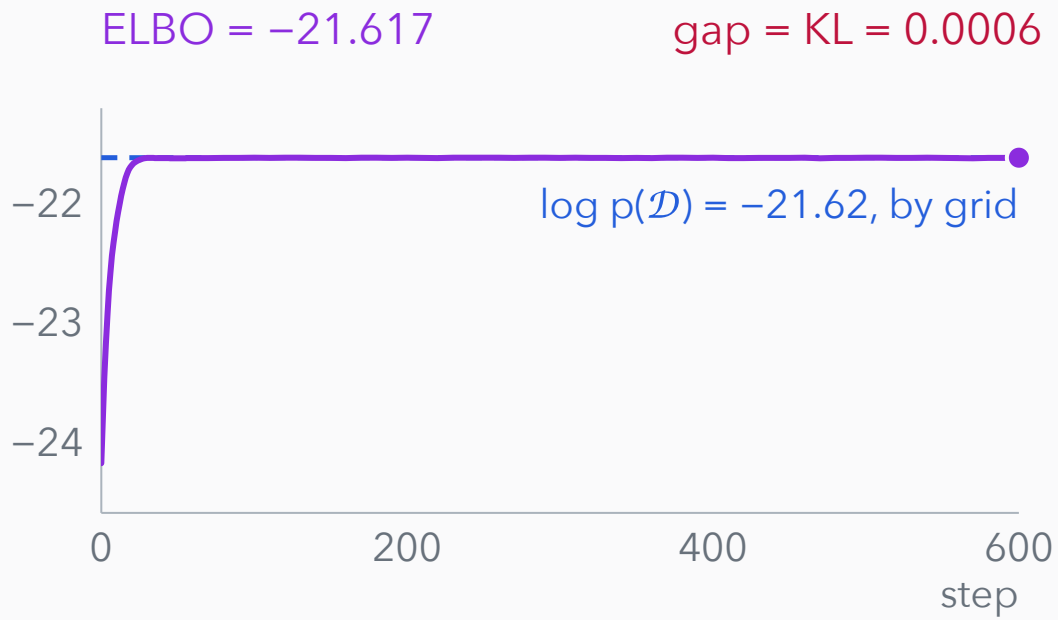


$$\mathrm{ELBO}(q) \le \log p(\mathcal{D})$$

As the ELBO rises the gap shrinks, never below zero: the ELBO is a lower bound on log p(𝒟).

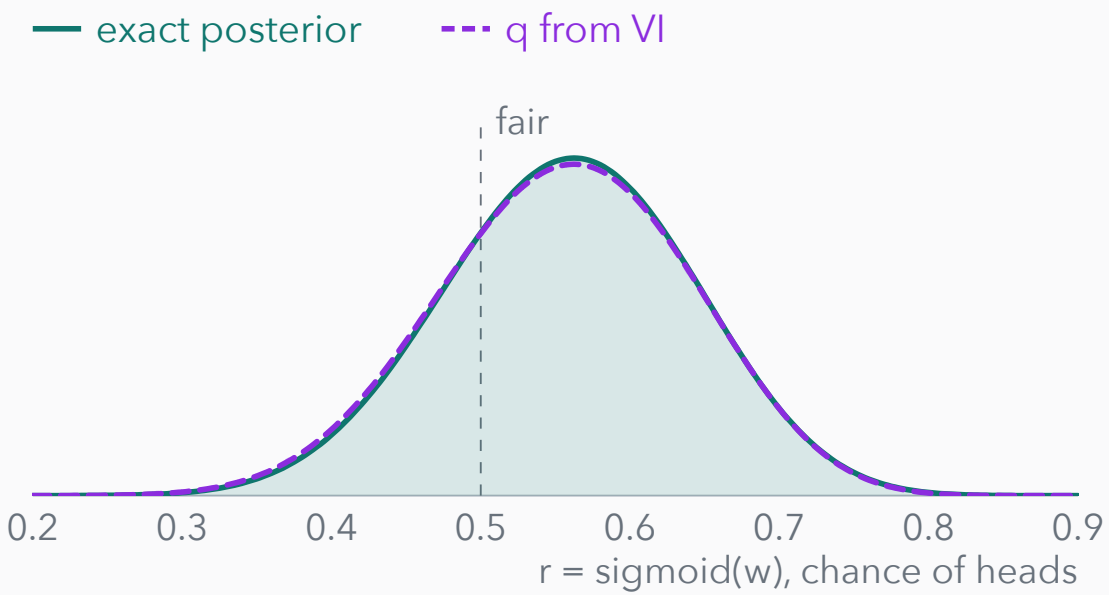
Back to the coin with no formula





> Within about 30 steps q sits on the posterior. The ELBO stops 0.0006 below log p(𝒟): that gap is KL(q ‖ posterior).

So, is the coin fair?

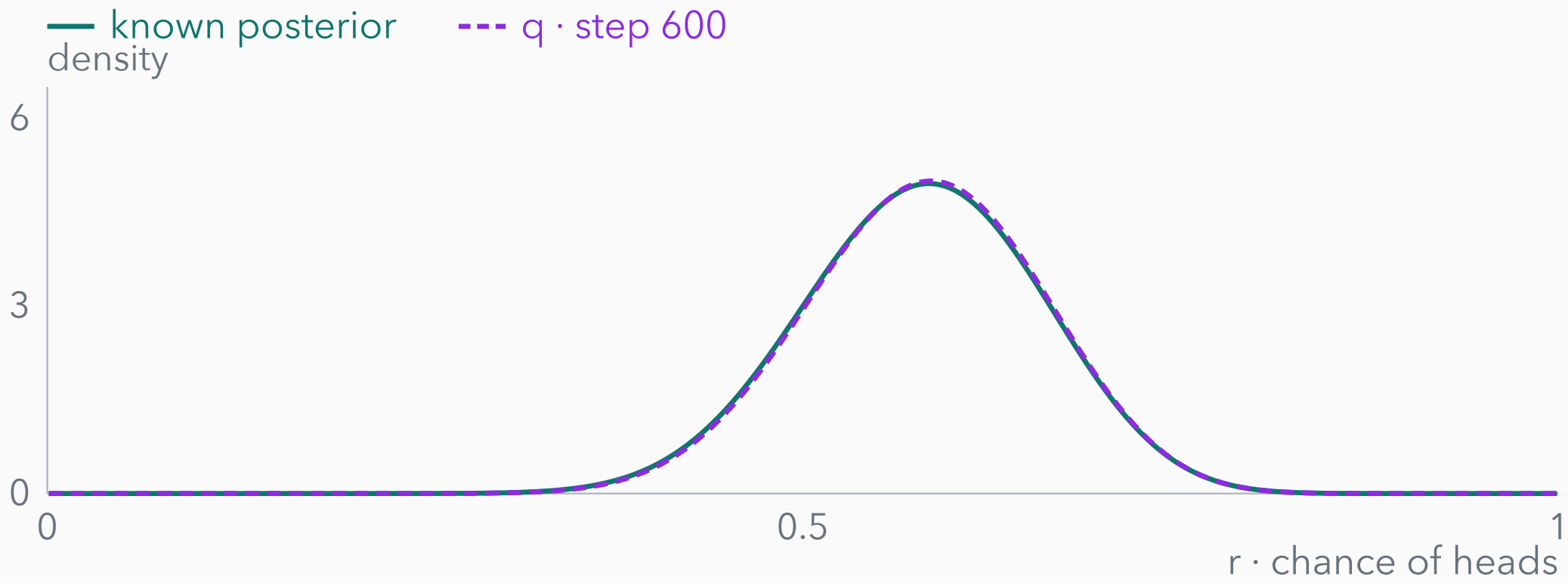


Map q back to the chance of heads, $r = \text{sigmoid}(w)$.

	$P(r > 0.5)$
q, from VI	0.75
exact, by grid	0.76

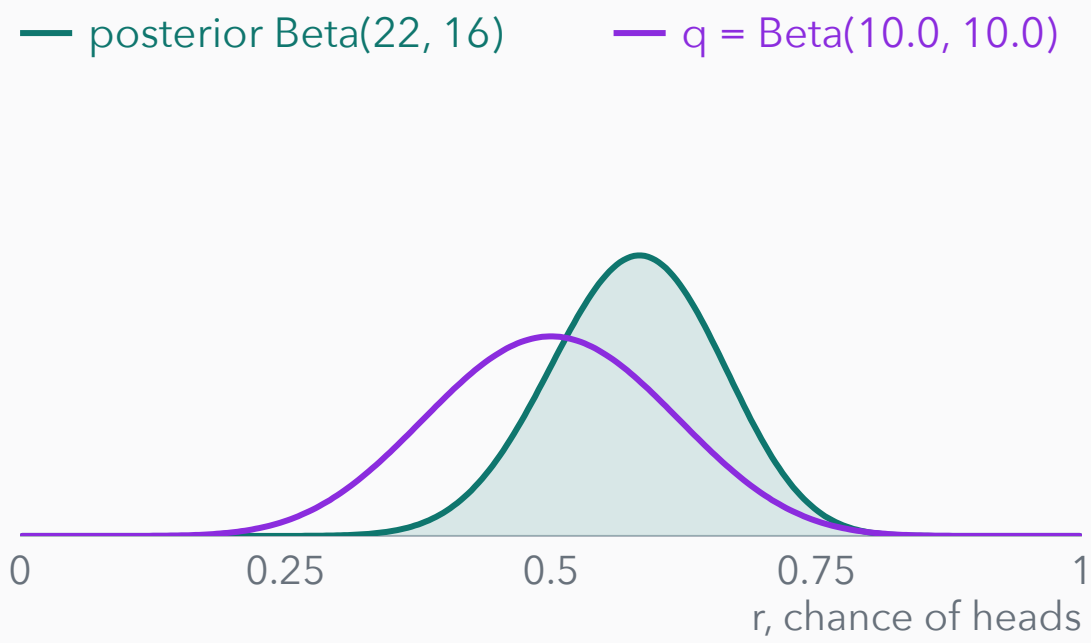
> P(r > 0.5) ≈ 0.75: probably biased towards heads, but a fair coin is still quite plausible. That answers the opening vote.

A check with an answer key: the Beta coin

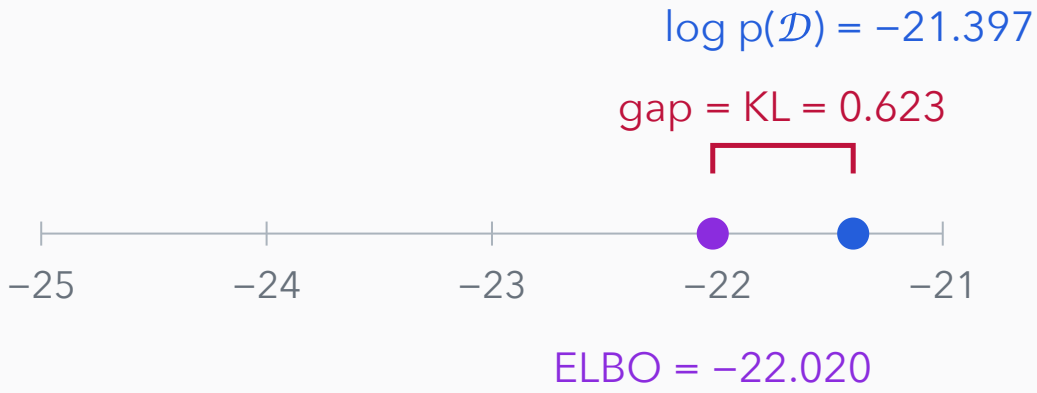


After 600 steps: Beta(22.4, 16.2), with KL ≈ 0.0002 to the true posterior.

Move q yourself: ELBO + KL never changes



One fixed total



−22.020 + 0.623 = −21.397

> Drag a and b. The ELBO and the KL trade off, and their sum stays at log p(𝒟) = −21.40.

Five obstacles, five tools

obstacle	tool	in the VAE (Lecture 3)
“closer” had no number	KL(q‖p)	the KL term in the loss
a score cannot improve q	gradient descent on q’s parameters	training the encoder
no formula for the KL	Monte Carlo averages	one draw per image
no gradient through a draw	u = μ + σε, rsample()	the reparameterisation trick
the posterior is unknown	maximise the ELBO	the VAE’s training loss

Five obstacles, five tools. In Lecture 3 a VAE uses all five at once, with one Gaussian q per image.

A sensor has an unknown constant bias

$$u \sim \mathcal{N}(0, 1), \qquad y_i \mid u \sim \mathcal{N}(u, 0.5^2)$$

$$p(\mathcal{D}, u) = p(u) \prod_i p(y_i \mid u)$$

$$\max_{\psi} \mathbb{E}_{q_\psi(u)}[\log p(\mathcal{D}, u) - \log q_\psi(u)]$$

> Maximise the ELBO over q's parameters. The same tools apply.

Generative models create this inference problem again

A hidden description can generate an observation.

Next: learn representations for images, then ask whether they can generate new ones.

> Lecture 2 builds autoencoders; Lecture 3 returns to VI with an encoder that outputs q for each image.

One literal update of the mean

$$\sigma = 2 \text{ fixed}: \qquad L(\mu) = \text{const} + \frac{(\mu - 1)^2}{32}$$

$$\frac{\partial L}{\partial \mu} = \frac{\mu - 1}{16} = -0.0625 \ \text{ at } \mu = 0$$

$$\mu^{+} = 0 - 4(-0.0625) = 0.25: \quad \text{KL } 0.349 \to 0.336$$

Take one step with η = 4. What is the best KL with σ stuck at 2?

Where does the Gaussian KL formula come from?

$$\log \frac{q(u)}{p(u)} = \log \frac{s}{\sigma} + \frac{(u-m)^2}{2s^2} - \frac{(u-\mu)^2}{2\sigma^2}$$

$$\mathbb{E}_q[(u-m)^2] = \sigma^2 + (\mu - m)^2, \quad \mathbb{E}_q[(u-\mu)^2] = \sigma^2$$

$$\mathrm{KL}(q\|p) = \log \frac{s}{\sigma} + \frac{\sigma^2 + (\mu-m)^2}{2s^2} - \frac{1}{2}$$

Where does the final −1/2 come from?

KL ≥ 0 by Jensen's inequality

$$\mathrm{KL}(q\|p) = -\mathbb{E}_q\left[\log \frac{p(u)}{q(u)}\right]$$

$$\geq -\log \mathbb{E}_q\left[\frac{p(u)}{q(u)}\right]$$

$$= -\log 1 = 0$$

What makes the final expectation one?

Five draws can give a negative KL estimate

u_i	$\log q(u_i) - \log p(u_i)$
5.623	−0.232
0.120	0.269
−5.536	−0.751
2.705	0.217
−2.254	0.030

$$\widehat{\mathrm{KL}}_5 \approx -0.093, \qquad \mathrm{KL}(q\|p) \approx 0.069$$

> Does this contradict KL ≥ 0?

The two PML candidates, scored by Monte Carlo

candidate	MC estimate ± 1 s.e.	numerical integral
$N(3, 1^2)$	9.54 ± 0.03	9.49
$N(3, 4.5^2)$	45.02 ± 0.21	45.15

The narrow candidate scores far better. Both sit away from the mixture's mass, so both are poor.

> What role does the numerical integral play?

A fixed-noise finite difference checks the path

```
q = Normal(mu, sigma)
u = q.rsample((S,))
loss = (q.log_prob(u) - p.log_prob(u)).mean()
```

derivative	autograd	finite difference
μ	−0.062466	−0.062466
σ	−0.372223	−0.372223

What must stay fixed on both sides of the finite difference?

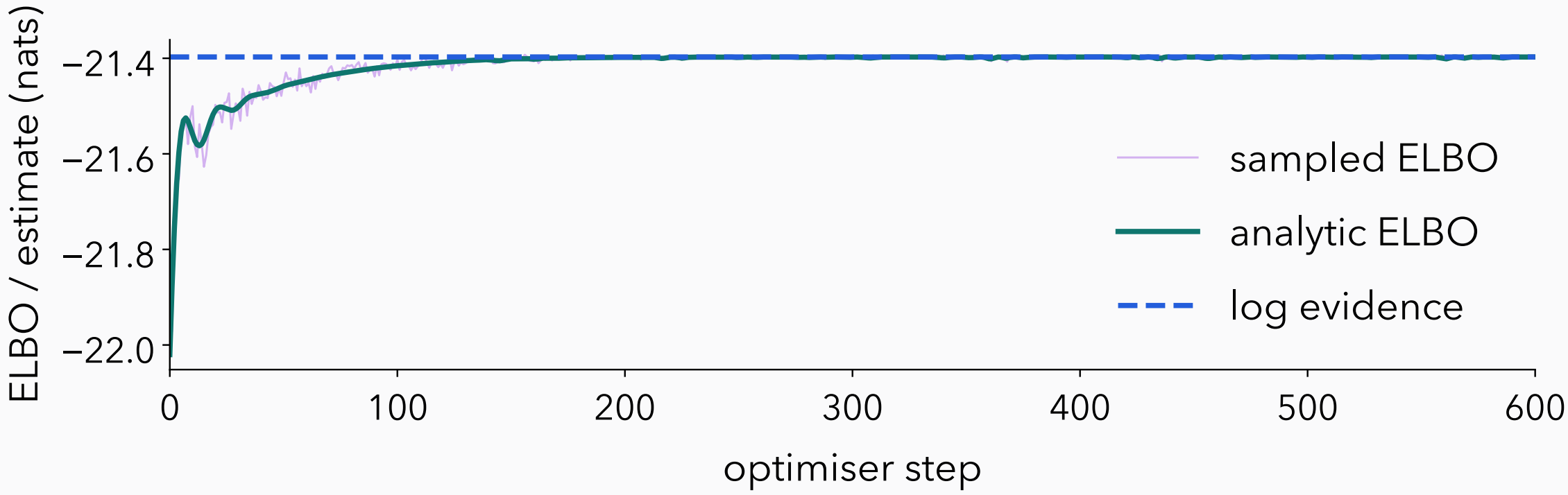
Pathwise gradients are one route, not the only one

$$\nabla_\psi \mathbb{E}_{q_\psi}[f_\psi(u)] = \mathbb{E}_{q_\psi}[f_\psi(u)\nabla_\psi \log q_\psi(u) + \nabla_\psi f_\psi(u)]$$

A score-function estimator accounts explicitly for the changing sampling distribution.

> Does this identity require u = μ + σε?

A noisy ELBO estimate can cross the evidence



162 of 600 sampled estimates lie above the known evidence in this run.

> Which object remains a lower bound?

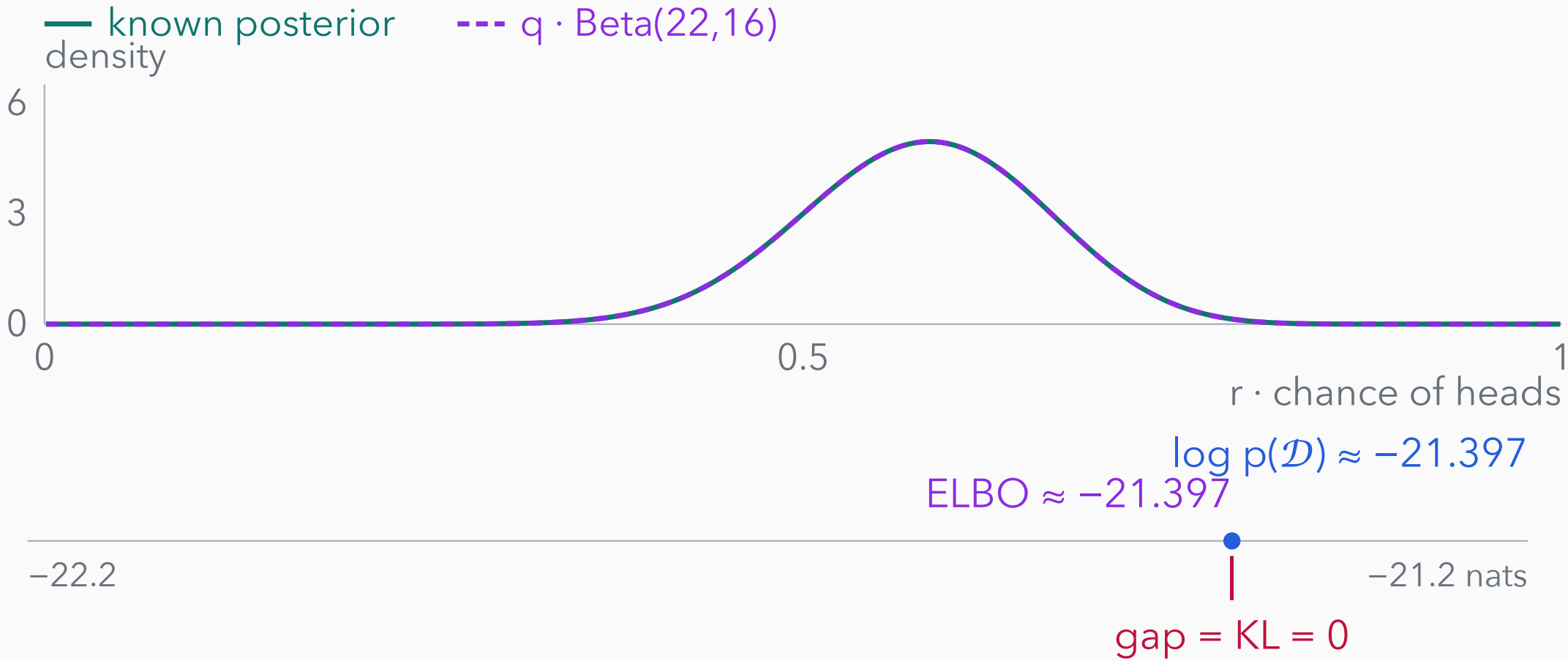
A higher ELBO does not prove a zero gap

$$\mathrm{ELBO}(q_{\mathrm{new}}) > \mathrm{ELBO}(q_{\mathrm{old}})$$

The family may not contain the posterior, and optimisation may stop short.

> Does a stable loss establish q = p(u | $\mathcal{D}$)?

Check the fixed total with three Beta candidates



What happens at the exact posterior?

One Gaussian per weight: Bayesian neural networks

$$q(\mathbf{w}) = \prod_j \mathcal{N}(w_j;\, \mu_j, \sigma_j^2)$$

$$\text{ELBO} = \mathbb{E}_q\big[\log p(\mathcal{D} \mid \mathbf{w})\big] - \text{KL}\big(q(\mathbf{w}) \,\|\, p(\mathbf{w})\big)$$

> How would you train it?

A mean must say which dimension it averages

object	shape	correct reduction
S scalar Beta draws	[S]	mean over S
S d-dimensional joint log densities	[S]	mean over S
S factorised coordinate log densities	[S, d]	sum over d, then mean over S

MultivariateNormal.log_prob already sums the event dimension. Do not sum the sample axis as well.

> What goes wrong if you sum over S before averaging?

An ordered sequence and a count have different evidence

$$p(H = 17 \mid r) = \binom{30}{17} r^{17}(1-r)^{13}$$

observed object	log evidence
one ordered sequence	−21.397
only the head count	−2.796

> Does this constant change the posterior or its KL gap?

Check the sampled objective with analytic Beta moments

$$\mathbb{E}_q[\log r] = \mathrm{digamma}(a) - \mathrm{digamma}(a+b)$$

$$\mathbb{E}_q[\log(1-r)] = \mathrm{digamma}(b) - \mathrm{digamma}(a+b)$$

Insert these into expected log likelihood + expected log prior − expected log q.

> How was the identity checked non-circularly?